INGENIERÍA WEB · SEGUNDO PREVIO · REV. 2.0

Manual técnico de auditoría e infraestructura



Colección de videojuegos 2D en HTML5 Canvas (Pixel Gol — Fútbol 2D Atari — y Neon Pong Turbo) integrada al Portafolio Web Profesional: lógica de animación, colisiones, arquitectura JavaScript, persistencia con LocalStorage y despliegue serverless.

ESTUDIANTE
Dainel Ramirez

ASIGNATURA
Ingeniería Web

UNIVERSIDAD
Universidad Simón Bolívar

CIUDAD / FECHA
Cúcuta, 29/10/2026

RESUMEN DEL ENTREGABLE

Qué contiene este manual

Contenido

03	Arquitectura del sitio y de archivos
04	Pixel Gol: requisitos del Fútbol 2D Atari
05	Diagrama de flujo: Game Loop a 60 FPS
06	Diagrama de flujo: detección de colisiones
07	Arquitectura de funciones JavaScript (Pixel Gol)
08	Neon Pong Turbo integrado en la sección Arcade
09	Persistencia de marcadores (LocalStorage + JSON)
10	Modo oscuro / claro y diseño responsivo
11	Rendimiento: 60 FPS sin fugas de memoria
12–14	Git/GitHub · Despliegue HTTPS · Lighthouse móvil
15	Extensibilidad para la prueba presencial

Stack técnico

HTML5 semántico · CSS3 · variables · JavaScript ES6+ (clases) · Canvas 2D · requestAnimationFrame · LocalStorage API · JSON · Web Audio API · Pointer / Touch events · Bootstrap Grid · GitHub Pages / Vercel

4
páginas del sitio integradas

2
juegos 2D en Canvas evaluados

60
FPS medidos en el Game Loop

URL pública

`https://_____.github.io/`

Completar con la URL del despliegue (página 13).

ARQUITECTURA DE INFORMACIÓN

Estructura del sitio y de archivos

```
# Parcial1_Dainel_Ramirez/
index.html            Inicio · marca personal · sobre mí
blog.html             Proyectos (diseño, motion, edición)
juegos.html           Sección Arcade (4 juegos)
contacto.html         Formulario validado con JS
script.js             Menú, animaciones, filtros, scroll
js/theme.js           Alternador modo oscuro / claro
css/styles.css        Guía visual + tema claro
assets/img · video    Imágenes y videos optimizados
juegos/
├─ pixel-gol/
│  ├─ futbol.html       Fútbol 2D Atari
│  ├─ css/style.css
│  └─ js/game.js        844 líneas, 11 clases
├─ neon-pong/index.html Ping-Pong
├─ sky-aces/  · deadzone/   (3D extra)
```

Navegación común

Las cuatro páginas comparten el mismo `<header>` fijo (Inicio · Marca personal · Proyectos · Juegos · Sobre mí · Contacto), el botón de tema y el mismo pie de página. En móvil (≤ 700 px) el menú se convierte en hamburguesa con `aria-expanded`.

Integración de los juegos

`juegos.html` presenta cada juego como tarjeta con portada, etiquetas y botón "Jugar". Los juegos abren en pestaña nueva, tienen enlace "◀ Juegos" de regreso y leen el mismo tema guardado (`portafolio-tema`), así respetan la guía visual del portafolio.

Carga en el navegador

`theme.js` se carga en el `<head>` para aplicar el tema antes de pintar (sin parpadeo). Imágenes con `loading="lazy"`, videos diferidos con `IntersectionObserver`, y los juegos no usan librerías externas.

VIDEOJUEGO 1 · FUTBOL.HTML

Pixel Gol: Fútbol 2D estilo Atari



Partido en curso: cancha, arcos con red, 2 jugadores, 2 arqueros, balón, marcador, reloj y barra de velocidad (VEL).

Requisitos del previo

- **Canvas 2D 320×200** escalado con píxeles nítidos (estética Atari 2600)
- **Bucle con requestAnimationFrame** a 60 FPS y delta-time limitado
- **Velocidad progresiva:** +1 px/s por segundo y +3,5 px/s por toque (105 → 260 px/s)
- **Goles en tiempo real** en el HUD, reloj y gol de oro en empate
- **Colisiones exactas** en subpasos: bandas, fondo, red, 4 postes, jugadores y arqueros
- **Modo 1 jugador vs IA** (fácil · normal · difícil) que persigue el balón
- **Modo 2 jugadores local:** P1 con W/A/S/D y P2 con flechas
- **Pausa** con Espacio o botón, e instrucciones visibles
- **Récords e historial** en LocalStorage con botón de reinicio

Controles

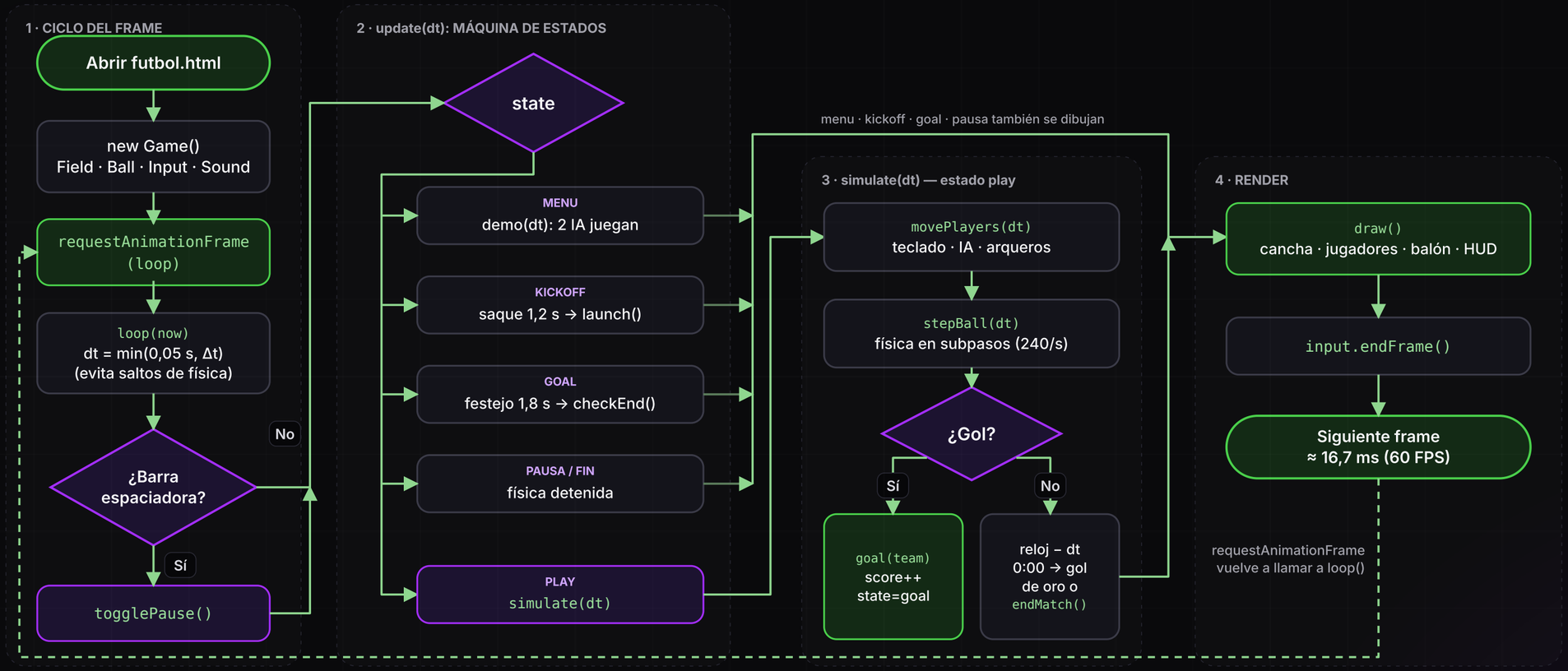
P1: W/A/S/D · P2: flechas
Espacio: pausa / continuar
Teléfono: joystick y botón II

Reglas

Gana quien llegue a 3, 5 o 7 goles o quien vaya ganando al acabar el tiempo (2, 3 o 5 min). Empate: gol de oro.

DIAGRAMA DE FLUJO 1

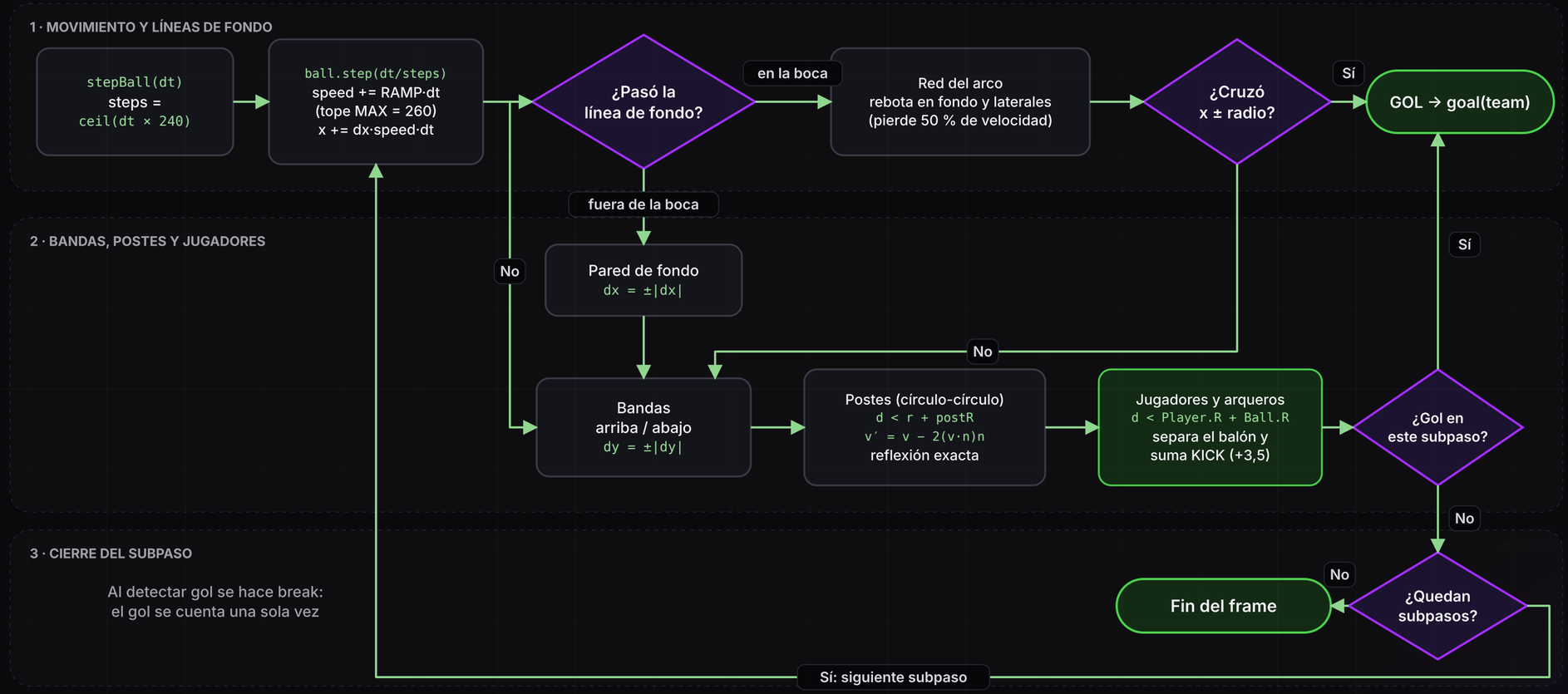
Lógica de animación: Game Loop



Un solo `requestAnimationFrame` encadena cada frame: calcula `dt` (máx. 0,05 s para que una pestaña inactiva no "teletransporte" el balón), actualiza según el estado del juego y dibuja. La pausa congela la física pero sigue dibujando, así la pantalla nunca se bloquea.

DIAGRAMA DE FLUJO 2

Detección de colisiones



La física se divide en subpasos (`ceil(dt × 240)`, ≈ 4 por frame): a máxima velocidad el balón avanza ~1 px por subpaso, así nunca "atraviesa" un poste o a un jugador (efecto túnel). Los postes usan reflexión vectorial `v′ = v − 2(v·n)n`.

ARQUITECTURA DE FUNCIONES · JS/GAME.JS

Clases ES6 de Pixel Gol

Game
Orquesta el juego: estados, bucle, reglas, marcador y fin del partido.

`loop(now) · update(dt) · simulate(dt)`
`movePlayers() · stepBall() · goal(team)`
`checkEnd() · endMatch() · togglePause()`
`draw() · drawHUD() · drawPause()`

Ball
Posición, dirección normalizada y velocidad progresiva del balón.

`reset() · launch(dir) · setDir(x,y)`
`step(dt) → wall | post | goalL | goalR`
`draw(ctx)`

Field
Medidas de la cancha, boca del arco y 4 postes.

`left/right/top/bottom · mouth · posts[ ]`
`draw(ctx) · pixelCircle()`

Player
Jugador de campo: movimiento suave y choque con el balón.

`drive(dir, dt) · update(dt, field)`
`collide(ball, game) · draw(ctx)`

Goalkeeper
Hereda de Player. Predice dónde cruzará el balón su línea y despeja.

`think(dt, ball) · update() · collide()`

AIController
IA del modo 1P: se coloca detrás del balón y lo empuja al arco rival.

`LEVELS {facil, normal, dificil}`
`update(dt, ball)`

Records
Persistencia: récords e historial en JSON (LocalStorage).

`load() · history() · register(match)`
`save() · reset()`

Input · Joystick
Teclado (W/A/S/D, flechas, Espacio) y joystick táctil.

`axis(player, mode) · take(code)`
`endFrame()`

UI · Sound
Menús, resultado, tabla de récords, JSON y sonidos con Web Audio.

`showResult() · renderRecords() · download()`
`kick() · post() · goal() · whistle()`

Composición: `Game` contiene `Field`, `Ball`, 4 `Player` (2 de campo + 2 `Goalkeeper`), `Input` y `Sound`; usa `AIController` solo en modo 1P y llama a `Records.register()` al terminar cada partido.

VIDEOJUEGO 2 · INTEGRACIÓN ARCADE

Neon Pong Turbo



Partido con botón PAUSA visible, marcador, rachas, multiplicador y línea de controles en pantalla.

Funciones principales

`loop(t)`	Bucle con requestAnimationFrame
`update(dt)`	Movimiento de paletas y bolas
`paddleHit()`	Rebote según punto de impacto; +5,5 % de velocidad por golpe
`ai() · predict()`	IA en 3 niveles que predice el rebote en las bandas
`score()`	Puntos, rachas y multiplicador ×5
`pause() · resume()`	Tecla P/Esc, botón en pantalla y pausa automática al cambiar de pestaña
`saveMatch()`	Guarda el partido en `npt_history`
`resetStats()`	Reinicia récord e historial

Modos: 1 jugador vs CPU (W/S o flechas) y 2 jugadores (W/S y ↑/↓). Gana el primero a 7.

PERSISTENCIA DE MARCADORES

LocalStorage en formato JSON

Claves utilizadas

CLAVE	CONTENIDO
`pixelgol_records`	Objeto con récords máximos
`pixelgol_historial`	Arreglo de los últimos 50 partidos
`pixelgol_settings`	Modo, nivel de IA, goles y tiempo
`npt_hi`	Récord de puntos de Neon Pong
`npt_history`	Historial de Neon Pong (100 partidos)
`portafolio-tema`	"dark" o "light"

Flujo de datos

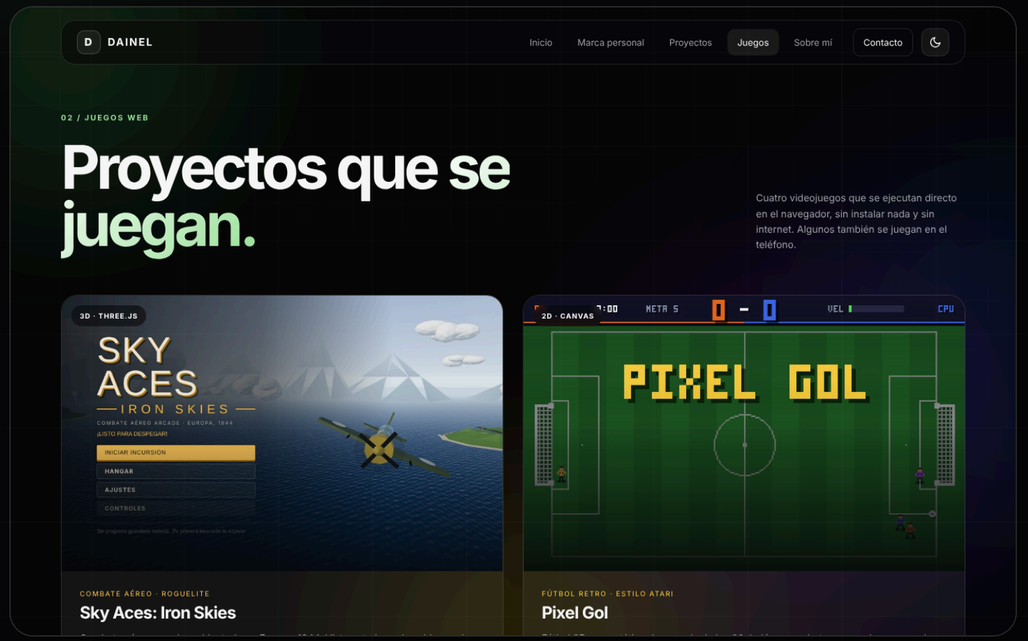
Fin del partido → `Records.register(match)` → `JSON.stringify()` → `localStorage.setItem()`. Al abrir la página: `getItem()` → `JSON.parse()` → tabla de récords e historial. Todo va en `try/catch` por si el navegador bloquea el almacenamiento. El botón **Reiniciar récords** pide confirmación y llama a `removeItem()`.

```
// localStorage["pixelgol_historial"][0]  (ejemplo)
{
  "fecha": "2026-10-29T20:48:10.512Z",
  "modo": "1P",
  "dificultad": "normal",
  "marcador": { "p1": 5, "p2": 3 },
  "ganador": "P1",
  "duracionSeg": 142.6,
  "golesParaGanar": 5,
  "golDeOro": false,
  "goles": [
    { "equipo": "P1",  "segundo": 11.4 },
    { "equipo": "CPU", "segundo": 37.9 }
  ],
  "golMasRapido": 11.4
}

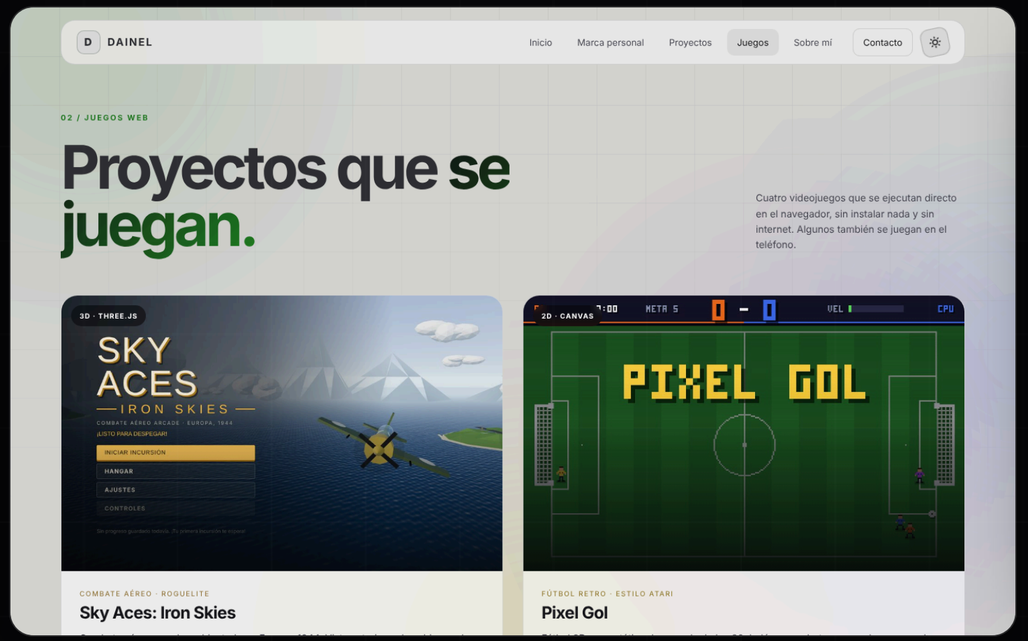
// localStorage["pixelgol_records"]
{ "partidos": 12, "victoriasVsIA": 7, "derrotasVsIA": 4,
  "mejorRacha": 3, "mayorGoleada": 4,
  "golMasRapido": 6.2, "golesTotales": 71 }
```

GUÍA VISUAL

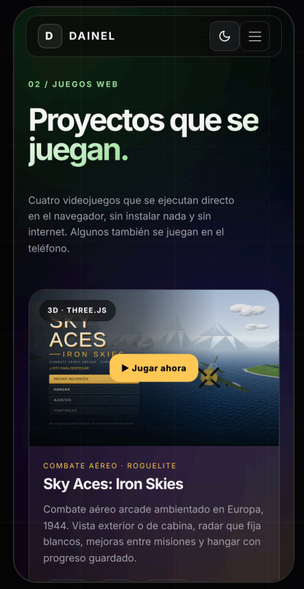
Modo oscuro / claro y responsivo



Modo oscuro (por defecto, identidad de la marca).



Modo claro: mismas variables CSS redefinidas en `[data-theme="light"]`.



Móvil 390 px.

Alternador

Botón ☾ /☀ en el menú de las 4 páginas y en los dos juegos. `theme.js` cambia `data-theme` en `<html>`, actualiza `aria-label` y `aria-pressed`, y guarda la elección.

Sincronización

La preferencia se comparte entre páginas y pestañas con el evento `storage`: si cambias el tema en el portafolio, el juego abierto también cambia.

Responsivo

Breakpoints a 900 y 700 px, menú hamburguesa, Canvas escalado por `aspect-ratio`, joystick táctil y aviso para girar el teléfono.

NOTA CRÍTICA · GAME LOOP

Rendimiento a 60 FPS sin fugas de memoria

60,5
FPS medidos con requestAnimationFrame (Chromium, partido en curso)

1
solo bucle activo: no hay `setInterval` ni bucles duplicados

240
subpasos de física por segundo para colisiones exactas

0
librerías externas en los juegos 2D

Decisiones de rendimiento

- **Resolución interna baja** (320×200 / 384×216) escalada por CSS: muy pocos píxeles por frame
- **dt limitado a 0,05 s:** sin saltos al volver de otra pestaña
- **Objetos reutilizados:** balón, jugadores y cancha se crean una vez por partido
- **Estela y partículas con tope fijo** (4 posiciones / límite de bolas), sin crecimiento infinito
- **Pausa automática** en Pong al perder el foco (`blur` / `visibilitychange`)

Sin bloquear el hilo de renderizado

- Cálculos en O(n) con n = 4 jugadores; sin bucles de espera
- LocalStorage solo se escribe al terminar un partido, nunca dentro del frame
- Sonidos sintetizados con Web Audio (sin descargar archivos)
- Eventos de teclado guardan estado en un `Set`; el bucle lo lee una vez por frame
- `URL.revokeObjectURL()` tras descargar el JSON (libera memoria)

CONTROL DE VERSIONES

Evidencia Git / GitHub

+

Captura del historial de commits

GitHub → repositorio → pestaña "Commits" (o el historial en GitHub Desktop), mostrando los commits del Segundo Previo.

+

Captura del repositorio

Vista principal del repositorio en GitHub con las carpetas del portafolio.

Commits sugeridos

```
feat: juego Pixel Gol (futbol.html)
feat: historial y estadísticas en Neon Pong
feat: alternador modo oscuro/claro
fix: main duplicado en juegos.html
docs: manual técnico segundo previo
```

DESPLIEGUE CONTINUO SERVERLESS

Publicación con HTTPS

Captura del panel de despliegue

Vercel → Deployments (estado "Ready") o GitHub → Settings → Pages ("Your site is live at …").

Captura del sitio con el candado HTTPS

Barra del navegador mostrando la URL pública y el candado.

Pasos (GitHub Pages)

- Subir el repositorio con GitHub Desktop (Push origin)
- Settings → Pages → Branch `main` / root
- Marcar **Enforce HTTPS**
- Probar `juegos/pixel-gol/futbol.html` en la URL pública

AUDITORÍA DE RENDIMIENTO

Google Lighthouse versión móvil

Captura del reporte de Lighthouse (Mobile)

Chrome → F12 → Lighthouse → Device: Mobile → Analyze page load, sobre la URL publicada. Meta: 85 o más en Rendimiento.

Resultados (completar)

PÁGINA	REND.	ACCES.	PRÁCTICAS	SEO
index.html	__	__	__	__
juegos.html	__	__	__	__
blog.html	__	__	__	__
contacto.html	__	__	__	__

Optimizaciones aplicadas

- Imágenes con `loading="lazy"` y videos diferidos
- Bootstrap solo con la rejilla (sin CSS sin usar)
- Un solo `<main>` por página y `aria-label` en botones
- `meta description` y `lang="es"` en todas las páginas

PREPARACIÓN DE LA PRUEBA PRESENCIAL

Dónde añadir una mecánica sorpresa

Puntos de extensión del código

MECÁNICA	DÓNDE SE PROGRAMA
Viento	`Ball.step()`: sumar una fuerza a `dx/dy` y volver a normalizar con `setDir()`
Turbo con energía	`Player.drive()` + barra en `drawHUD()`; tecla leída en `Input`
Obstáculos	Lista en `Field`; choque igual a los postes (círculo-círculo) en `Ball.step()`
Gol gana	`checkEnd()`: terminar con `score ≥ 1`

```
// Ejemplo: viento en Ball.step(dt)
const viento = game.viento;  // {x: 0.3, y: 0}
if (viento) {
  this.setDir(
    this.dx + viento.x * dt,
    this.dy + viento.y * dt
  );
}

// Ejemplo: modo "Gol gana" en checkEnd()
if (this.settings.golGana &&
    (this.score[1] || this.score[2])) {
  this.endMatch(); return true;
}
```

Como cada responsabilidad vive en su clase, una regla nueva toca uno o dos métodos sin alterar el bucle de animación ni los 60 FPS.